Merge Sort VS Heapsort

Gizelle De La Torre, Lifu Erhart, Henry Owen, Mohamed Eljack
gizelle.delatorre@wsu.edu, lifu.Erhart@wsu.edu, henry.owen@wsu.edu,mohamedalamen.elack@wsu.edu
CPTS 350, Dr. Asma Jodeiri Akbarfam

***Abstract*—This project looks at the performance of three sorting algorithms, Merge Sort, Heap Sort, and Double Pop Heap Sort. Although all three have the same theoretical time complexity, they do not always perform the same in practice. The goal of this project is to see how these algorithms behave as the input size increases and which algorithm performs better in terms of runtime. To keep the comparison fair, each algorithm was tested using randomly generated data of different sizes. Each test was run multiple times, and the average runtime was recorded to make the results more reliable. The results were then compared using graphs. So far, the results suggest that all three algorithms follow the expected pattern, but Merge Sort tends to perform faster than Heap Sort. The Double Pop version shows some improvement over regular Heap Sort, but not enough to outperform Merge Sort. These results show that even when algorithms have the same time complexity, their actual performance can still be different.**

I. Introduction

As data sizes increase, the efficiency of sorting algorithms becomes more important. In this project, we compare the performance of three sorting algorithms, Merge Sort, Heap Sort, and a variation called Double Pop Heap Sort. All three algorithms have the same theoretical time complexity of O(n log n), which suggests they should perform similarly as input size increases. However, algorithms with the same time complexity can behave differently due to factors such as implementation details, constant factors, and the types of operations they perform. The objective of this project is to analyze how these algorithms perform as input size increases and to determine which one is more efficient in terms of runtime. So, to ensure a fair comparison, each algorithm is tested using the same input data across multiple trials, and the average runtime is recorded. This allows for a more accurate evaluation of performance. By comparing these algorithms, this project aims to highlight the difference between theoretical efficiency and real-world performance and to provide a better understanding of how algorithm design affects execution time.

II. Background

The merge sort algorithm sorts an array by dividing it into multiple sub-arrays. It splits the list of items in half over and over to create separate subproblems that are simpler to solve. Once solved, the subproblems are then merged back together. The time complexity for this algorithm is O(n log n). Though, it does need about O(n) extra space for the temporary lists that are used when merging.

Despite also being used for sorting, the heap sort algorithm works in a different way. It builds a list called a max-heap from the elements, then removes the biggest element and moves it to the end. This procedure is done repeatedly in order to help establish the final sorted array. Heap sort does have the same time complexity as merge sort, which is O(n log n). Although, it has the advantage of not needing much extra space and only about O(1) extra memory. However, the main problem with this algorithm is that it jumps around in memory a lot, which can cause it to slow down.

Double-pop heap sort is a lesser known variation of heap sort. This algorithm is similar to the original heap sort, but double-pop removes two elements from the list at the same time instead of only one. After it takes out the elements, it finds the next biggest item and takes that out too before putting the list back in order [2]. This reduces the number of operations it has to run in order to get the final sorted list. Double-pop heap sort also shares the same time complexity of O(n log n), but the procedures it does can cause a difference in performance.

All three algorithms take about the amount of time O(n log n) but they use different amounts of space and work differently with memory, which affects how fast they really are. That is what we are trying to measure.

III. Related Work

Cormen et al. [1] and other researchers wrote extensively about merge sort and heap sort. We used their work to guide our implementations of these algorithms. However, their work mainly focuses on theory and does not include testing to compare actual runtime performance. The idea of double-pop heap sort came from a discussion on the Computer Science Stack Exchange website [2], where it was suggested that removing two items at a time could reduce the number of heap operations needed. There is some sample code available, but there has not been much testing done to measure how fast it performs. LaMarca and Ladner [3] showed that memory behavior can significantly affect algorithm performance, and that heap sort tends to perform worse in this area compared to other O(n log n) algorithms. This is something we expect to see in our results. Our project tests all three algorithms using randomly generated data with different input sizes. Most other research either ignores the double-pop variation or focuses only on theory without actually testing its performance.

IV. System Design / Methodology

A. Merge Sort

`merge_sort(A, left, right)` recursively splits the array at the midpoint. After both halves are sorted, an early-exit check is performed: if `A[mid] <= A[mid+1]`, the two halves are already in order and `merge` is not called.

Otherwise, `merge` interleaves the two halves using temporary arrays `L` and `R`. Runtime is $O(n \log n)$; auxiliary space is $O(n)$.

B. Heap Sort

`heap_sort(A)` calls `build_heap(A, n)` to construct a max-heap in $O(n)$ time. It then iteratively swaps `A[0]` (the maximum) with the last unsorted element and calls `heapify` to restore the heap property, reducing heap size by one each iteration. Overall complexity is $O(n \log n)$; space is $O(1)$.

C. Double Pop Heap Sort

`dp_heap_sort(A)` shares `build_heap` and `heapify` with `heap_sort`. In each main iteration, it places the root at the sorted end, then identifies the larger child of the new root and places it at the next sorted position, calling `heapify` on the affected subtree and then on the root. A standard heap sort fallback handles the final three elements. Complexity is $O(n \log n)$; space is $O(1)$.

D. Input Generators

Eight input types were used to capture a range of practical scenarios:

- **Integers**: unique random samples from $[0, 10n)$
- **Floats**: uniform random values from $[0, 10n)$
- **Strings**: random 6-character lowercase strings
- **Random**: unique integer samples (same as Integers, used as baseline)
- **Sorted**: $[0, 1, \ldots, n-1]$
- **Reverse Sorted**: $[n, n-1, \ldots, 1]$
- **Nearly Sorted**: sorted array with $\lfloor n/20 \rfloor$ random element swaps (≈5% disorder)
- **Many Duplicates**: random integers from $[0, n/10)$, producing many repeated values

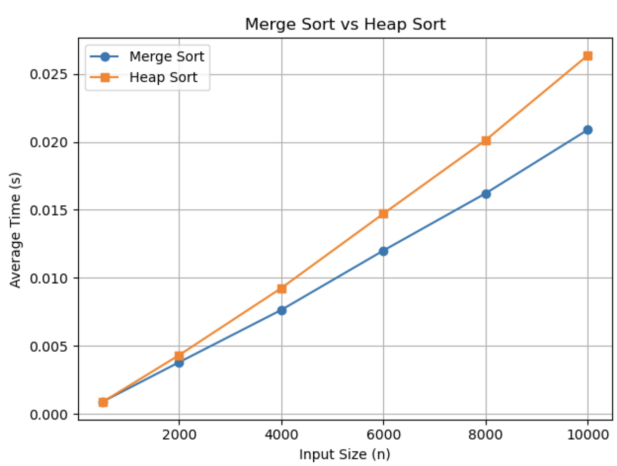

Fig. 1. Runtime comparison of Merge Sort and Heap Sort showing performance as input size increases.

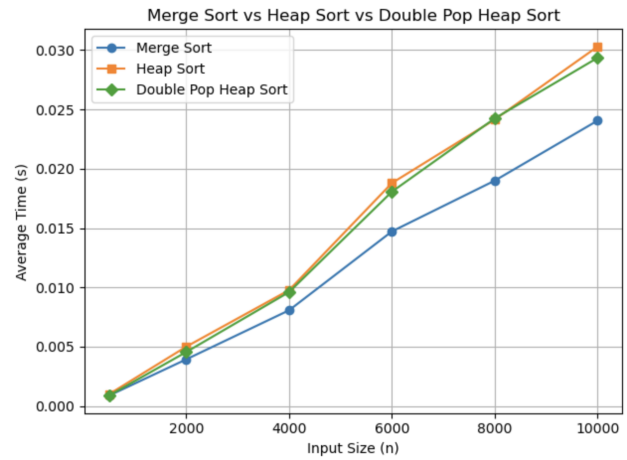

Fig. 2. Runtime comparison of Merge Sort, Heap Sort, and Double Pop Heap Sort for smaller input sizes.

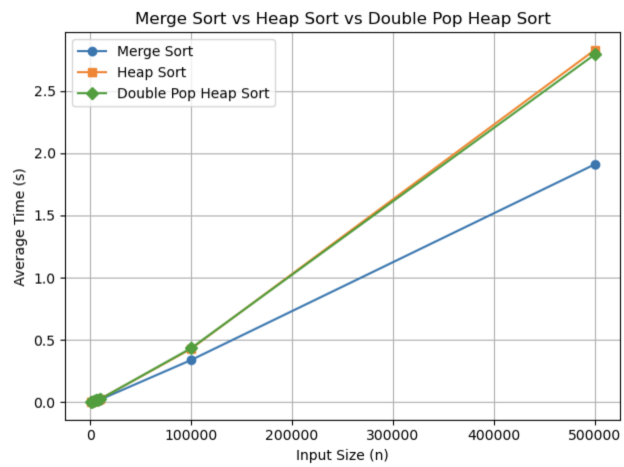

Fig. 3. Runtime comparison of Merge Sort, Heap Sort, and Double Pop Heap Sort for larger input sizes.

V. Implementation / Experiment Setup

Runtime was captured using `time.perf_counter`, which provides high-resolution wall-clock timing. Peak memory usage is measured using Python's `tracemalloc` module, which tracks all heap allocations during a function call. The peak is recorded in kilobytes using `tracemalloc.get_traced_memory()`. This allows direct comparison of the memory footprints of all three algorithms independently of runtime. To test our algorithms, we ran them on input arrays of random values in sizes of 500, 2000, 4000, 6000, 8000, 10000, 100000, and 500000. All three algorithms were given the same randomized values in each test. Ten tests were run for each input size and times were recorded, and then the average of all ten times was plotted against input size. To perform our experiments, we used VS code with the Jupyter notebook extension running a python 3.10.12 kernel.

VI. Preliminary Results or Progress

Here are our three graphs that were created when running the multiple tests of merge sort, heap-sort, and double-pop heap-sort. In these graphs you can see how and when the performance of each one begins to diverge from one another and demonstrate a clear idea of which ones perform at a faster pace. As well as, since they all share the same time complexity of O(n log n), that does not change and the arcs show the representation. There is a noticeable change between the three algorithms and how despite merge sort performing at a faster pace, the graphs show how much more memory it uses.

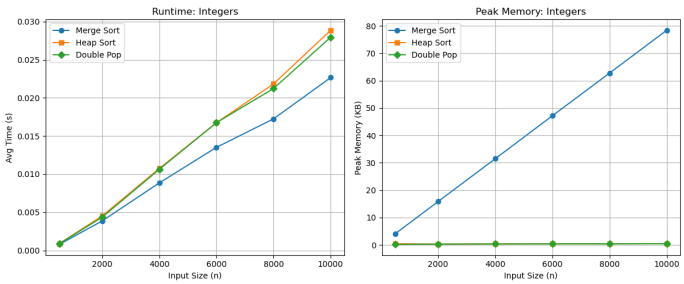

Fig. 4. Demonstrates how all three algorithms work in terms of runtime and memory with integers.

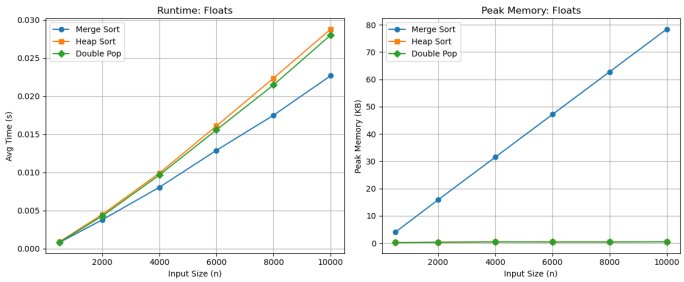

Fig. 5. Demonstrates all three algorithms with their runtime and memory when dealing with floats.

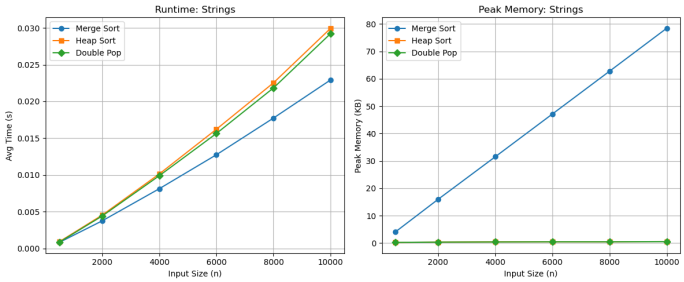

Fig. 6. Demonstrates all three algorithms with their runtime and memory peak when handling string input.

VII. Discussion / Next Steps

For the next stage of this project, we plan to expand our analysis beyond just runtime to get a more complete understanding of algorithm performance. One step is to test each algorithm on different types of input data, such as already sorted arrays, reverse sorted arrays, and arrays with many duplicate values. This will help us see if performance changes depending on the structure of the data and allow us to identify the best and worst-case scenarios.

We also plan to compare the memory usage of each algorithm to better understand the tradeoff between speed and space efficiency. Right now, we are only looking at runtime, so including memory usage will give a more complete comparison. In addition, we plan to improve the accuracy of our results by increasing the number of tests runs and keeping testing conditions consistent. We will also improve how results are presented by using clearer graphs and tables to make comparisons easier to understand.

VIII. Conclusion

We decided to take an empirical approach for our project, and this gave us the ability to take three sets of algorithms and test them against one another. The tests involved merge sort, heap sort, and double-pop heap-sort, and running them against the same exact problem. We paid close attention to see which algorithm seemed to finish sorting an array faster than the others or if all three worked at the same pace because they share a time complexity of O(n log n).

Our results demonstrated that double pop heap-sort seemed to have a slightly better performance when compared to the original heap-sort. The difference in speed was not a huge amount, but it is noticeable. This is because when the double-pop is running it works by removing two elements from the array, while regular heap-sort will only remove one. Removing two elements reduces the amount of heap operations the algorithm has to perform and therefore produces a shorter sorting time.

When all three were compared against each other the results varied and a clear difference in performance was easy to notice. The results showed that both double-pop heap-sort and the regular heap-sort were still slightly similar in performance as mentioned, but merge sort had them both beat in performance. Merge sort had taken the less amount of time in order to take an unsorted array that contained random data and have it sorted in a shorter amount of time. This is because merge sort works using the divide-and-conquer method, which allows the algorithm to perform more simpler operations when sorting and performing recursive. Although, merge sort has been proven to use more memory throughout our tests compared to the other two algorithms. Therefore, merge sort would be the more beneficial algorithm to use when sorting an array.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms. Cambridge, MA, USA: MIT Press, 2009.

[2] "Can this double-pop heapsort variation speed up sorting on average?" Computer Science Stack Exchange, 2019. [Online]. Available: https://cs.stackexchange.com/questions/108968

[3] A. LaMarca and R. E. Ladner, "The influence of caches on the performance of sorting," in Proc. 8th Annu. ACM-SIAM Symp. Discrete Algorithms (SODA), 1997, pp. 370–379.

Appendix A
Appendix (Optional)

Additional diagrams, extended configurations, or code snippets.